

CAHIER DES CHARGES TECHNIQUE

Architecture Globale "Full-Stack Edge" : Rendu SSR & Back-End Micro-Workers

Statut : Spécifications Définitives | **Version** : 3.0.0 (Intégration Front-End) | **Environnement** : Cloudflare Pages, Workers, D1, Astro, Hono

1. Contexte et Synthèse Architecturale

Ce document définit l'architecture complète d'un système de gestion de contenu (CMF) 100 % Edge-natif. Afin de combiner les performances SEO d'un rendu côté serveur (SSR) et la scalabilité infinie d'une infrastructure sans serveur, nous adoptons l'approche "**Full-Stack Edge**".

Le Front-End (rendu de l'interface utilisateur) et le Back-End (API et base de données) sont tous deux déployés sur le réseau mondial de Cloudflare. Ils communiquent exclusivement via le réseau interne (Service Bindings), éliminant ainsi la latence HTTP publique et assurant un typage strict de bout en bout.

2. Pile Technologique Complète (La Stack)

2.1 Couche Front-End (Rendu SSR à l'Edge)

- Framework** : **Astro** (ou Next.js Edge Runtime). Choisi pour son architecture orientée "Zero-JS par défaut" (Island Architecture), idéal pour les sites de contenu (blogs, e-commerce, médias).
- Hébergement** : **Cloudflare Pages** avec intégration des *Pages Functions* pour exécuter le serveur SSR d'Astro directement sur les nœuds Edge.
- Client API** : `hono/client` (RPC), permettant d'importer les types du Back-End pour une auto-complétion parfaite.

2.2 Couche Back-End (Architecture Micro-Workers)

- Logique Métier** : **Cloudflare Workers** orchestrés par le framework **Hono**.
- ORM & Base de données** : **Drizzle ORM** couplé à **Cloudflare D1** (SQLite distribué).
- Topologie** : Découpage en micro-services internes (Auth Service, Content Service) non exposés sur l'Internet public.

3. Topologie du Réseau et Communication (Service Bindings)

L'avantage majeur de cette architecture réside dans la suppression du goulot d'étranglement réseau classique entre un serveur Front (ex: Vercel) et une base de données distante (ex: AWS RDS).

Composant	Rôle Exclusif	Exposition Publique
Astro SSR (Cloudflare Pages)	Génère le HTML final. Gère les requêtes utilisateur (Navigateur -> Edge).	OUI (Domaine principal)
Worker de Contenu (Hono)	Exécute la logique métier Drizzle et interroge la BDD D1.	NON (Binding Interne uniquement)
Worker d'Authentification	Valide les sessions, génère les JWT, gère les droits (RBAC).	NON (Binding Interne uniquement)

Sécurité accrue : Les Workers Back-End ne possèdent pas de route HTTP publique (pas de routes dans `wrangler.toml`). Ils sont invisibles depuis l'extérieur et insensibles aux attaques DDoS directes, car seul le serveur Astro peut les invoquer.

4. Implémentation du Typage de Bout en Bout

4.1 Le Worker Back-End (Fournisseur de Types)

Le service de contenu définit ses routes avec un typage strict et exporte son interface :

```
// packages/content-worker/src/index.ts
import { Hono } from 'hono';
import { db } from '../db'; // Instance Drizzle
import { contentNodes } from '../schema';

const app = new Hono();

const routes = app.get('/api/articles', async (c) => {
  const articles = await db.select().from(contentNodes).all();
  // Le type inféré est renvoyé automatiquement
  return c.json({ data: articles });
});

// Exportation vitale pour le Front-End
export type ContentAPI = typeof routes;
export default app;
```

4.2 Le Front-End Astro (Consommateur via RPC Interne)

Dans un composant Astro exécuté côté serveur (SSR), nous appelons le Worker interne. Aucun `any` n'est utilisé, tout changement dans la base de données remonte jusqu'au template HTML.

```

---
// apps/frontend/src/pages/index.astro
import { hc } from 'hono/client';
// Importation exclusive des TYPES (le code métier back n'alourdit pas le front)
import type { ContentAPI } from '../../../../packages/content-worker/src/index';

// Astro permet d'accéder aux bindings configurés sur Cloudflare Pages
const env = Astro.locals.runtime.env;

// Création du client RPC en branchant directement la fonction fetch du binding
// Cela by-passe le réseau HTTP public (Latence < 2ms)
const client = hc<ContentAPI>('http://interne', {
  fetch: env.CONTENT_WORKER.fetch.bind(env.CONTENT_WORKER)
});

// Appel fortement typé : TypeScript sait exactement ce que contient 'data'
const res = await client.api.articles.$get();
const { data: articles } = await res.json();
---

<html>
  <head><title>Mon CMF Edge</title></head>
  <body>
    <h1>Derniers Articles</h1>
    <ul>
      {/* Auto-complétion active sur article.title et article.slug */}
      {articles.map((article) => (
        <li><a href={`/${article.slug}`}>{article.title}</a></li>
      ))}
    </ul>
  </body>
</html>

```

5. Stratégie de Déploiement Monorepo

Pour que cette symbiose typée fonctionne, l'ensemble du code doit résider dans un **Monorepo** (géré par npm workspaces ou Turborepo).

- **Commande de build global** : Un seul push sur la branche main déclenche l'action CI/CD.
- **Ordre de déploiement** : Les modifications de schémas (Migrations D1) s'exécutent en premier. Ensuite, les Micro-Workers Back-End sont mis à jour via `wrangler deploy`. Enfin, Cloudflare Pages build et déploie le projet Astro.

6. Bilan Économique et Performances (vs Monolithe Classique)

Métrique	Monolithe VPS (Drupal/WP)	Full-Stack Edge (Astro/Hono)
Coût d'entrée mensuel	~ 20 € à 50 € (VPS + Sauvegardes)	0 € (Free Tier Cloudflare extrêmement large)
TTFB (Time to First Byte)	200ms - 800ms (selon la distance)	< 40ms (Rendu SSR exécuté au plus proche de l'utilisateur)
Résilience au trafic (Spike)	Faible (Crash de la base ou du serveur HTTP)	Totale (Auto-scaling par Cloudflare)
Complexité de développement	Modérée (Tout-en-un PHP/MySQL)	Élevée (Architecture distribuée, TypeScript strict, Monorepo)

Conclusion : Ce cahier des charges valide une architecture state-of-the-art. Elle sacrifie la simplicité d'installation initiale d'un CMS traditionnel au profit d'une sécurité maximale (Back-end invisible), d'une performance SEO optimale (SSR Edge) et d'une sécurité typée absolue entre l'interface utilisateur et la base de données.